PROJECT & PATENT BRIEF — UPDATED (REVIEW-2)

Personalized Multi-Modal Integrity Scoring for Live Coding Exams

SNEHA – 24BIT0228
ABHISHEK MITRA – 24BIT0477
MEHUL KUMAR SRIVASTAVA – 24BIT0251

Course: BITE308L · Vellore Institute of Technology
Guide: [Faculty guide's name]

Status as of 28 October 2026

> This document updates the Review-1 brief to the implemented system. The prototype is complete up to the research stage: capture, baseline, all four signals, the reviewer workflow, the study tooling and the evaluation harness are built and tested. The one research result so far is the reproduction of the published keystroke model on public data. Accuracy on real students will be measured in an ethics-approved study; no such result is claimed here. Section 2 lists every change from Review-1.

1. Abstract

Online coding assessments struggle to verify whether submitted code was written by the examinee or produced by a Large Language Model (LLM), a second person or a script and then inserted. Common approaches rely on plagiarism comparison, population-level AI-code detectors and raw paste logs; they ignore each student's own way of typing and writing code, and paste logs are easily bypassed.

This project builds and evaluates a **personalized multi-modal integrity scoring system** for live coding exams. A self-hosted web exam environment with an embedded code editor records, inside the editor only, every key press and release with its timing, every edit, paste and copy, focus changes, test runs and code snapshots, in an encrypted, append-only, hash-chained event log. Each student's personal baseline is built from three short practice tasks and then tested by a held-out baseline check. Four signals are compared with that baseline: **keystroke rhythm** (a TypeNet-style LSTM encoder trained with triplet loss), **writing-process** features, **code-structure (AST)** features and **clipboard and focus** behaviour. They are combined into an Integrity Confidence Score $S \in [0, 1]$ by a sign-constrained logistic regression; low scores go to a human reviewer and nothing is decided automatically.

Results to date: on the public Aalto dataset (168,593 people), our keystroke encoder reproduces all ten published TypeNet results within pre-registered acceptance bands (for example 1.57% equal error rate with five enrollment samples, against 1.7% published), 15–20 times better than two statistical baselines (20.6–26.3%). The full system, a reviewer dashboard, the study tooling and an evaluation harness are implemented and covered by about 430 Python and 160 browser tests. Detection accuracy on real students will be measured in a study of 30–50 consenting students once ethics approval is granted.

2. Changes since Review-1

The design was refined during implementation. Each change is recorded with its reason in the project's decision log (D-numbers).

Aspect	Review-1 plan	Implemented	Why
Capture	Browser extension (Chrome / VS Code plugin)	Self-hosted web exam app with an embedded Monaco editor; no extension in v1	Full control of the editor (no find-replace or auto-insert), nothing to install on study laptops (D-001, D-023)
Keystroke model	Siamese network, cosine distance	TypeNet-style 2-layer LSTM, triplet loss; Euclidean distance to the student's prototype	A published, verifiable model that could be reproduced first (D-029, D-033)
Pre-training data	"150 open-source typing datasets"	Aalto 136M Keystrokes: 168,593 people, licence checked	One large, documented dataset with a published benchmark
AST divergence	Kullback–Leibler divergence	Jensen–Shannon divergence on problem-normalised, smoothed node distributions, plus style z-scores	KL is unbounded and undefined for unseen node types; JSD is bounded in [0, 1]
Fusion	Shallow MLP	Logistic regression with every distance weight constrained ≤ 0; MLP only if a pre-registered ablation shows a gain	An MLP overfits 30–50 students; the constraint guarantees more anomaly never raises S (D-018)
Fusion inputs	ΔK, ΔA, P_vol, P_freq	7 distances (keystroke, process, AST histogram, AST style, external-paste share, external-paste size, focus loss) + 2 availability flags	Adds the writing-process signal; handles missing modalities (D-013)
Enrollment	2–3 practice modules	3 practice tasks with the code shown, plus a held-out baseline check	Nobody gets stuck; the check tests the baseline (D-035, D-038)

Aspect	Review-1 plan	Implemented	Why
Tamper resistance	Not covered	Server-side reconciliation of edits against key presses; hash-chained log	Client telemetry can be forged (D-026)
Student view	Not specified	Students see no report; reviewers see everything	Lead's decision (D-049)
Webcam	Not planned	Opt-in demo feature for test accounts; reviewers can view the video; not analysed. The head and eye movement panel is a video demonstration with example output	Demonstrates a planned extension (D-048)

Table 1. Changes from the Review-1 design to the implemented system.

3. Problem Statement

Existing automated proctoring approaches have clear limits when separating a student's own coding from outside help:

- **No individual baselines.** Detectors judge code against population-level notions of "AI-like" code and ignore each student's natural style and typing rhythm, so strong students who write clean, idiomatic code risk being flagged.
- **Cold start.** Personal behavioural models normally need long histories, which short courses and single assessments do not have, and collecting them raises privacy concerns under the Digital Personal Data Protection Act, 2023.
- **Weak single signals.** On public data we measured two simple statistical typing baselines at 20.6–26.3% equal error rate, against 1.2–4.1% for a deep encoder (Section 12).
- **Bypassable telemetry.** Paste logs are evaded by auto-typers, by retyping from a second screen, or by forging the browser's telemetry.

4. Motivation

Code-generation tools have outpaced conventional integrity controls. Cohort-comparison plagiarism tools such as MOSS (based on document fingerprinting [6]) do not catch code that is unique to each submission, as LLM output usually is. Recent work separates human from LLM-written code using stylometric and AST features [8, 9], and keystroke dynamics have been used to separate bona fide from LLM-assisted writing of text [7]. These techniques have not been combined into a real-time, per-student system for programming exams.

A short, low-stakes enrollment (three practice tasks) builds a personal baseline with little data, and fusing typing, writing process, code structure and clipboard behaviour targets the cases each signal misses alone. The output is evidence for a person, never an automatic verdict, which keeps false flags in human hands.

Patent angle. Section 3(k) of the Indian Patents Act, 1970 excludes computer programs per se, so any claims are framed around specific technical mechanisms: the held-out baseline check with a capped personal threshold that cannot disable detection; server-side reconciliation of edit deltas against key events; and the sign-constrained fusion of distance signals. Patentability and the applicable guidelines for computer-related inventions are to be confirmed with the university's IPR cell [VERIFY]. The Aalto dataset is licensed for non-commercial research; whether patent-support documents count as commercial use is an open question for the team.

5. Literature Survey and Gap Identification

5.1 Prior work

- **Keystroke dynamics.** Early work established typing rhythm as a biometric [4]; a benchmark of 14 anomaly detectors on a fixed password showed the limits of statistical features [3]. TypeNet trained an LSTM with triplet loss on the Aalto data and reached about 1–5% equal error rate on short English sentences [1, 2]. All of this is natural-language typing, not code.
- **Keystrokes and LLM assistance.** An IJCB 2024 study used a TypeNet-based Siamese network to tell bona fide writing from ChatGPT-assisted writing in 40 participants [7]. It addresses essays, not programming.
- **Code stylometry and AI-code detection.** AST-based features can attribute programmers [5] and separate human from LLM code [8, 9]. These judge the final code against a population and are sensitive to renaming and refactoring.
- **Few-shot behavioural biometrics.** Triplet and Siamese embeddings allow enrollment from a few samples in other behavioural modalities [10, 11].

5.2 Research gaps

- **Cold-start personalization for code:** a per-student baseline from a few short tasks, tested before it is trusted.
- **Modality disconnect:** no real-time fusion of low-level typing, the writing process, code structure and clipboard behaviour.
- **Domain adaptation:** keystroke models trained on natural language are untested on syntax-heavy code typing.
- **Honest evaluation:** few studies report every attack, including failures, with confidence intervals over users and a fixed false-flag rate.

6. Survey Table

Table 2 summarises representative prior work. Figures are those reported by the sources; entries marked [VERIFY] still need checking against the original paper.

Ref.	Method	Data	Reported result	Limitation for our setting
[4]	Keystroke dynamics as a biometric (statistical)	Small user groups	Feasibility of typing biometrics	Fixed or short text; no code
[3]	Benchmark of 14 anomaly detectors	51 subjects typing one password	Best detectors around 10% EER [VERIFY]	Fixed password, not free text
[1, 2]	TypeNet: LSTM + triplet loss (checked)	Aalto: 168,593 people × 15 sentences	EER 1.2–4.5% (M=70, G=10…1)	English sentences in one sitting; not code
[7]	TypeNet-based Siamese network, bona fide vs ChatGPT-assisted	40 participants writing text	Discernible differences reported	Essays, not programming
[5]	AST-based code stylometry (random forest)	Google Code Jam authors	High attribution accuracy [VERIFY]	Final code only; needs several samples per author
[8]	AST and stylistic features, human vs ChatGPT code	Programming tasks	Distinguishing features reported	Population-level; no personal baseline
[9]	140 stylometric features, human vs GPT-4 code	CodeChef problems	F1 / AUC about 0.91	Final code only; sensitive to edits

Ref.	Method	Data	Reported result	Limitation for our setting
[6]	Winnowing fingerprints (MOSS)	Cohort submissions	Efficient copy detection	Unique LLM outputs evade it
[10]	Few-shot behavioural authentication (Siamese / triplet)	Mobile motion sensors	Up to 97% F1 with 3 shots	Different modality

Table 2. Representative prior work (sources checked or marked [VERIFY]).

7. Objectives and Status

Objective	What was built	Status
Client-side capture	Web exam app: policy-controlled Monaco editor; key down/up with timing, edits, pastes, copies, focus, test runs, snapshots; batches every 15 key presses or 5 s over WebSocket	Done
Few-shot personal baseline	3 practice tasks (code shown) → TypeNet embeddings of 70-key windows → personal prototype (≥ 3 windows); process and AST baselines; held-out baseline check	Done
Real-time multi-modal fusion	Scoring ticks; four signal families; evidence rules in use; sign-constrained fusion, smoothing and threshold implemented and tested end to end on synthetic data	Built; to be fitted on study data
Human-in-the-loop dashboard	Evidence report and typing chart, replay, hash-chain check, process table, AST jumps, webcam video, decisions with required notes, named reviewer accounts, access log	Done
Evaluation	Study protocol tooling, scripted attacks, data export and nested cross-validation harness with all planned analyses	Built; study awaits ethics approval

Table 3. Objectives from Review-1 and their status.

8. Design Methodology

8.1 Two phases

Phase	Steps
1. Enrollment (sitting 1, ~40 min)	Consent → device questions → 3 practice tasks with the code shown (typed, tested, submitted) → baseline check (one more typed task, held out) → personal baseline: keystroke prototype, process baseline, AST baseline, personal threshold if the check is confirmed
2. Live scoring (exam)	A scoring tick every 35 key presses (once a 70-key window exists), at every paste and at least every 30 s → signals over the trailing 120 s → evidence rules / fused score S → smoothing → reviewer dashboard. Nothing is decided automatically

Table 4. The enrollment and live-scoring phases.

8.2 Mathematical formulations

Keystroke features. For key i with down time d_i and up time u_i (as in TypeNet):

$$HL_i = u_i - d_i \quad PL_i = d_{(i+1)} - d_i \quad IL_i = d_{(i+1)} - u_i \quad RL_i = u_{(i+1)} - u_i$$

A window of M = 70 key presses, with each key code divided by 255 and latencies in seconds, is mapped by the encoder f to a 128-dimensional embedding.

Encoder training (triplet loss, margin α = 1.5). For an anchor a and positive p from the same person and a negative n from another person:

$$L = \max(0, \|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + \alpha)$$

Keystroke distance. The student's prototype is the mean embedding of the practice windows; each exam window w is compared with it:

$$p_u = (1/G)\,\Sigma_g\, f(w_g) \qquad d_{ks}(w) = \|f(w) - p_u\|_2$$

A window is "beyond the line" when d_ks exceeds the 95th percentile of same-person distances measured on held-out validation subjects (or the student's own line from a confirmed baseline check, capped at 1.25 times that).

Process distance. For the 14 process features p_k, the student's practice baseline b_k and the pooled within-user standard deviation s_k:

$$d_{proc} = \sqrt{\,(1/K)\,\Sigma_k\,((p_k - b_k)/s_k)^2\,}$$

AST distributions. For a snapshot with named-node counts h(v), N nodes in total and a vocabulary V (Laplace $\alpha = 0.5$), the distribution is tilted by the per-problem reference R_p, and drift is the Jensen–Shannon divergence (base 2) from the student's baseline B_u:

$$P(v) = (h(v) + \alpha)/(N + \alpha|V|) \qquad Q(v) \propto P(v)/R_p(v) \qquad d_{ast,h} = JSD_2(Q_{exam}, B_u) \in [0, 1]$$

Four style features (nesting depth, identifier length, share of one-letter names, comment share) are z-scored per problem: $d_{ast,s} = \text{mean}_k\, |z_{exam,k} - z_{base,k}|$.

Integrity Confidence Score. With the seven distances x_j and two availability flags a_m, standardised on training data:

$$S = \sigma\Big(b + \Sigma_j\, w_j\, x_j + \Sigma_m\, v_m\, a_m\Big), \quad w_j \le 0, \quad \sigma(z) = 1/(1 + e^{-z})$$

Smoothing and the session decision. Each tick's score is smoothed on the logit scale ($\lambda = 0.5$), and a session is routed to review if its smoothed score ever falls below τ, set so that 5% of genuine validation sessions are flagged:

$$\ell_k = \lambda \cdot \text{logit}(S_k) + (1 - \lambda) \cdot \ell_{(k-1)} \qquad S_{session} = \min_k\, \sigma(\ell_k)$$

9. Architecture Modules

- **Module 1 — Capture (web exam app).** Monaco editor with a restricted policy (replace and code-inserting features are never loaded), event capture with timestamps, Python tests run locally in Pyodide, batches sent over WebSocket and stored with a server sequence number.
- **Module 2 — Personal baseline generator.** Practice windows → TypeNet embeddings → prototype; process and AST baselines; the baseline check with a capped personal threshold that never switches detection off.
- **Module 3 — Evidence and fusion engine.** Keystroke, process, AST and clipboard signals; server-side reconciliation of edits against key presses; evidence rules today; sign-constrained fusion with smoothing once fitted.
- **Module 4 — Reviewer dashboard.** Report with plain verdicts and a typing chart, replay, hash-chain check, process table, AST jumps, webcam video, decisions with required notes, named reviewer accounts and an access log.
- **Module 5 — Privacy and governance.** Consent with separate choices, pseudonyms, encryption at rest, withdrawal, retention job, and a guard that refuses real collection until ethics approval and governance values are in place.

- **Module 6 — Study and evaluation tooling.** Latin-square study plans, frozen AI-generated study material, scripted attacks in a real browser, data export and the nested cross-validation harness.

10. Datasets

ID	Data	Use	Status
D1	Aalto 136M Keystrokes: 168,593 people × 15 English sentences, millisecond timing [2]	Encoder training and the TypeNet reproduction	Used; licence: non-commercial research with attribution
D2	Our study: 50+ consenting adult students recruited so that 30–50 complete two sittings	Fine-tuning, fusion, calibration and evaluation	Tooling ready; awaits ethics approval
D3	Synthetic, labelled sessions and typists	Pipeline tests only; never reported as results	Used for testing

Table 5. Datasets.

Captured attributes: key down/up times and key codes (hold and three flight latencies), edits with positions and their cause, pastes with their source (inside or outside the editor), copies, focus changes, test runs, code snapshots, browser and device covariates, and, for opted-in test accounts only, webcam video without audio.

11. Implementation

- **Stack:** Python 3.11, FastAPI with WebSocket, PostgreSQL 16 (Docker), SQLAlchemy and Alembic; TypeScript, Vite and the Monaco editor; Pyodide for in-browser tests; PyTorch for the encoder (trained on an RTX 3050 laptop GPU); scikit-learn and SciPy; tree-sitter for Python parsing; pytest and Playwright for testing.
- **Storage:** event payloads encrypted with AES-256-GCM, bound to their row; append-only tables enforced by database triggers; every event extends a SHA-256 hash chain; keys kept outside the repository and any cloud-synced folder.
- **Key parameters:** 70-key windows (exam stride 35), a 120 s trailing horizon for process features, at least 3 practice windows for a baseline, a false-flag target of 5%; all settings live in configuration files, not code.
- **Webcam (demo feature):** opt-in for test accounts, no audio, recorded in 5 s chunks, encrypted and viewable by reviewers and admins. The video is not analysed; the head and eye movement panel is a video demonstration with example output for a planned analysis.

12. Results to Date

12.1 Keystroke encoder: TypeNet reproduction (real data)

The protocol and acceptance bands (within max(1 point, 50% relative) of each published value) were fixed before training. Split: 67,000 training, 1,000 validation and 1,000 test subjects. The only deviation, a learning rate of 0.005 instead of 0.05, was chosen on validation subjects before any test access.

M	G	Our mean per-subject EER	Published (TypeNet)	Acceptance band	In band
50	1	4.14%	5.4%	2.70–8.10	yes
50	2	2.52%	3.6%	1.80–5.40	yes

M	G	Our mean per-subject EER	Published (TypeNet)	Acceptance band	In band
50	5	1.55%	2.2%	1.10–3.30	yes
50	7	1.27%	1.8%	0.80–2.80	yes
50	10	1.07%	1.6%	0.60–2.60	yes
70	1	4.11%	4.5%	2.25–6.75	yes
70	2	2.65%	2.8%	1.40–4.20	yes
70	5	1.57%	1.7%	0.70–2.70	yes
70	7	1.33%	1.4%	0.40–2.40	yes
70	10	1.17%	1.2%	0.20–2.20	yes

Table 6. Equal error rate by window length M and number of enrollment samples G.

- **Training:** input batch normalisation, two LSTM layers of 128 units with dropout 0.5, triplet loss (margin 1.5), Adam, 512 triplets × 150 batches × 200 epochs, about 80 minutes per model; seeded and checkpointed.
- **Baselines on the same test subjects:** digraph 24.7–26.3% EER, sequence statistics 20.6–25.3%; the encoder is 15–20 times better.
- **Caveats:** with one global threshold, as our system uses, the EER is 2–3 times higher (3.42% at M=70, G=10); the sentences were typed in one sitting and are English text, not code; identity is not authorship.

12.2 End-to-end detection demonstration (scripted typists)

Scenario	Check	Exam distance (line 2.13)	Outcome
Same typist throughout	confirmed	0.96–1.40	No review flags
Another typist in the exam	confirmed	3.99–4.24	Flagged
Another typist in the baseline check	does not match	0.76–1.41	Review finding for the check

Table 7. The real app and model with two scripted typing profiles: a demonstration of the mechanism, not an accuracy measurement.

12.3 Evaluation pipeline

The complete evaluation (nested cross-validation, sign-constrained fusion, smoothing, threshold at 5% false flags, ablations, calibration and user-cluster bootstrap) runs end to end on synthetic data; no weight broke the sign rule. Synthetic numbers are not reported as results.

13. Testing and Quality

- About 430 Python tests: every feature on hand-built inputs with known answers, database rules (append-only triggers, encryption, erasure), the API and the evaluation harness.
- About 160 browser tests in Chromium, Firefox and WebKit: capture, editor policy, the full student and reviewer journey, the study protocol, withdrawal, webcam recording and the scripted-attack runner.
- 49 design decisions logged with their reasons; fixed seeds; every report names its data set, code version and configuration hash; synthetic data labelled in names and content; automated security review of every change, with all findings fixed.

14. Privacy, Ethics and Security

- **Data minimisation:** only activity inside the code editor; no microphone, screen or mouse recording; webcam only for opted-in test accounts.
- **Protection:** pseudonyms (reviewers never see names; the identity map stays offline with one custodian), encryption at rest, an access log of every read and decision, named reviewer accounts.
- **Control:** consent with separate choices, withdrawal at any time (erases all data, including traces in the access log), a retention job, adults (18+) only.
- **Ethics gate:** real-participant collection is switched off in the code and cannot be switched on without ethics approval, the governance values and named reviewer accounts.
- **Threat model:** the student controls the browser, so the server trusts nothing it is told; reconciliation, the hash chain and replay checks raise the cost of forging, while a fully rewritten client can still forge consistent telemetry.

15. Study Protocol and Evaluation Plan

- **Sittings:** sitting 1 (~40 min) for consent and enrollment; sitting 2 (at least a day later, ~60 min, same browser) for six short exam tasks.
- **Conditions (role-play, no consequences):** genuine ×2, A1 paste a solution, A4 copy-type it by hand, A5 edit AI code to look like one's own, A7 legitimate refactoring; order from a Williams Latin square. Scripted A2/A3 auto-typers in a real browser; A8 a different keyboard (false-flag test). Reviewers are blind to conditions.
- **Analysis fixed in advance:** user-disjoint nested cross-validation (5 outer folds); primary endpoints: detection per attack at a 5% false-flag rate, and fused score vs the best single signal; 95% confidence intervals by bootstrap over students (10,000 resamples); every attack reported, including failures; a ledger of every computation on test students.

16. Limitations

- Copy-typing with one's own hands (A4) and hand-edited AI code (A5) are expected to be hard to detect.
- Typing changes with a new keyboard, injury, fatigue or accessibility needs, which can cause false flags; every flag goes to a person.
- The baseline is copy-typed practice; composing an answer naturally has more pauses and corrections.
- A study of 30–50 students gives wide confidence intervals.
- If someone else completes the practice tasks, the baseline is theirs; enrollment should be supervised or low-stakes.
- Client-side data can be forged by a determined student with a rewritten browser client.

17. Project Status and Next Steps

Phase	Scope	Status
P0–P1	Specification, capture app, secure log, replay	Done, approved

Phase	Scope	Status
P2	Keystroke model on public data	Done, approved: 10/10 results within band
P3	Four signals, evidence report, fusion pipeline	Built; fusion fitted once study data exists
P4	Reviewer dashboard and workflow	Built
P5	Study tooling, evaluation harness, ethics	Tooling built; ethics approval next
P6	Paper and patent-support documents	After the study results

Table 8. Phase status.

Next steps: ethics application and governance values; a real-keyboard pilot with the team; data collection in two sittings; evaluation with the pre-registered endpoints; the paper and patent-support documents with every citation verified.

18. References

> The Review-1 brief listed 40 references. Spot checks could not locate several of them (for example the 2024 TPAMI entry on cold-start biometric profiles and the 2024 TIFS entry on Siamese networks for few-shot continuous authentication), so that list has been replaced by sources that were checked or are marked [VERIFY].

[1] Acien, A., Morales, A., Monaco, J. V., Vera-Rodriguez, R., & Fierrez, J. TypeNet: Deep learning keystroke biometrics. IEEE Transactions on Biometrics, Behavior, and Identity Science. arXiv:2101.05570 (checked against the arXiv text; journal volume and pages [VERIFY]).

[2] Dhakal, V., Feit, A. M., Kristensson, P. O., & Oulasvirta, A. (2018). Observations on typing from 136 million keystrokes. Proceedings of CHI 2018. https://doi.org/10.1145/3173574.3174220

[3] Killourhy, K. S., & Maxion, R. A. (2009). Comparing anomaly-detection algorithms for keystroke dynamics. IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2009). [VERIFY pages]

[4] Monrose, F., & Rubin, A. D. (2000). Keystroke dynamics as a biometric for authentication. Future Generation Computer Systems, 16(4), 351–359. [VERIFY]

[5] Caliskan-Islam, A., Harang, R., Liu, A., Narayanan, A., Voss, C., Yamaguchi, F., & Greenstadt, R. (2015). De-anonymizing programmers via code stylometry. USENIX Security Symposium. [VERIFY]

[6] Schleimer, S., Wilkerson, D. S., & Aiken, A. (2003). Winnowing: Local algorithms for document fingerprinting. ACM SIGMOD 2003. [VERIFY pages]

[7] Keystroke dynamics against academic dishonesty in the age of LLMs. IEEE International Joint Conference on Biometrics (IJCB 2024). arXiv:2406.15335. [VERIFY authors]

[8] Discriminating human-authored from ChatGPT-generated code via discernable feature analysis (2023). arXiv:2306.14397. [VERIFY authors and venue]

[9] Whodunit: Classifying code as human authored or GPT-4 generated — a case study on CodeChef problems (2024). arXiv:2403.04013. [VERIFY authors and venue]

[10] Fereidooni, H., et al. (2023). AuthentiSense: A scalable behavioral biometrics authentication scheme using few-shot learning for mobile platforms. NDSS 2023. [VERIFY]

[11] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. IEEE CVPR 2015.

[12] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

[13] Lin, J. (1991). Divergence measures based on the Shannon entropy. IEEE Transactions on Information Theory, 37(1), 145–151.

[14] The Digital Personal Data Protection Act, 2023 (India). [VERIFY sections]

[15] The Patents Act, 1970 (India), Section 3(k).

[16] Software: tree-sitter and tree-sitter-python 0.25; Monaco Editor; Pyodide; PyTorch; FastAPI; PostgreSQL (versions pinned in the repository).